

Big Bertha — Hardware Bridge

Build, run, and test guide · arduino@100.106.178.114 · 2026-06-27

CONTENTS

- | | | | |
|---|--------------------------------------|----|--|
| 1 | System overview | 2 | SSH into the board |
| 3 | Flash the MCU sketch | 4 | Run the bridge service |
| 5 | Build from source | 6 | Get IMU data |
| 7 | Control servos | 8 | PWM values explained |
| 9 | Check system status | 10 | File locations |

1. System Overview

Three layers communicate in sequence:

```
ROS 2 Node (your AI / control code)
|
| JSON lines over TCP port 50007
▼
hardware_bridge ← systemd service on Linux companion
|
| msgpack-rpc over unix socket
▼
arduino-router ← systemd service
|
| UART at 460800 baud
▼
STM32U585 sketch (MCU)
|
| I2C at 400 kHz
├─ MPU-6050 (0x68) IMU – pushes data at ~119 Hz
└─ PCA-9685 (0x40) 12× MG995 servo driver
```

Where output comes from: The MCU sketch reads the MPU-6050 and pushes IMU data up through arduino-router → hardware_bridge. Your code talks to hardware_bridge on port 50007 to get that data or send servo commands.

You never talk directly to the MCU or the router. Everything goes through port 50007.

2. SSH into the Board

FROM YOUR MACHINE

```
sshpass -p 'arduinoverse' ssh -o StrictHostKeyChecking=no arduino@100.106.178.114
```

Password: . Once inside, all commands below run on the board.

3. Flash the MCU Sketch

Do this whenever the sketch may have been overwritten by a teammate, or after a sensor power-cycle.

ON THE BOARD

```
# Step 1 – write the sketch file
python3 /tmp/write_sketch4.py

# Step 2 – build and flash (MCU resets automatically)
arduino-app-cli app start ~/ArduinoApps/hardware_bridge_app

# Step 3 – stop Docker (it starts automatically and steals the IMU route)
docker stop hardware_bridge_app-main-1

# Step 4 – wait ~10 s, then confirm MCU is connected
echo '{"cmd":"ping"}' | nc -q1 127.0.0.1 50007
# expected: {"ok":true}
```

Do not restart arduino-router after flashing. The running process keeps 460800 baud. Restarting it reverts to 115200 and the MCU will not connect.

VERIFY THE BAUD RATE

```
cat /proc/$(pgrep arduino-router)/cmdline | tr '\0' ' ' | grep -o 'baudrate [0-9]*'
# must show: baudrate 460800
```

4. Run the Bridge Service

The service starts on boot and runs automatically.

COMMON COMMANDS

```
systemctl status hardware-bridge.service      # is it running?
journalctl -fu hardware-bridge.service        # live logs
sudo systemctl restart hardware-bridge.service # restart after a rebuild
sudo systemctl start hardware-bridge.service  # if stopped
```

When the service starts it automatically stops the Docker container (`hardware_bridge_app-main-1`) so there is no route conflict.

5. Build from Source

Only needed if you change `hardware_bridge.cpp`.

Always use `make -j 1`. Other builds may be running on the board simultaneously.

FIRST TIME

```
cd ~/poc
mkdir -p build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release -Wno-dev
make -j 1 hardware_bridge
```

SUBSEQUENT BUILDS

```
cd ~/poc/build && make -j 1 hardware_bridge
sudo systemctl restart hardware-bridge.service
```

6. Get IMU Data

The MCU pushes IMU readings at ~119 Hz. hardware_bridge caches the latest frame. Query it over TCP on port 50007.

SINGLE READ

```
echo '{"cmd":"imu"}' | nc -q1 127.0.0.1 50007
```

RESPONSE FORMAT

```
{
  "ax": 0.311,           // accelerometer X (m/s²)
  "ay": -0.004,          // accelerometer Y (m/s²)
  "az": 10.479,          // accelerometer Z (m/s²) – ~9.81 at rest
  "gx": -0.040,          // gyroscope X (rad/s)
  "gy": -0.016,          // gyroscope Y (rad/s)
  "gz": 0.009,           // gyroscope Z (rad/s)
  "lin_acc_x": 0.311,    // ax (same)
  "lin_acc_y": -0.004,   // ay (same)
  "lin_acc_z": 0.672     // az - 9.80665 (gravity removed, robot upright)
}
```

CONTINUOUS STREAM (FROM YOUR MACHINE)

```
sshpass -p 'arduinoverse' ssh -o StrictHostKeyChecking=no arduino@100.106.178.114 \
  "while true; do echo '{"cmd":"imu"}' | nc -q1 127.0.0.1 50007; sleep 0.1; done"
```

FROM PYTHON / ROS 2

```
import socket, json

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(('100.106.178.114', 50007))
s.sendall(b'{"cmd":"imu"}\n')
data = json.loads(s.recv(1024))
s.close()
print(data['az']) # ~9.8 m/s² at rest
```

Real vs fake sensor: if `az` is exactly `9.80665` and everything else is exactly `0`, the I2C bus is stuck. Power-cycle the sensor board and reflash.

7. Control Servos

Send 12 raw PWM off-counts (one per joint). The bridge forwards them to the PCA-9685 in a single I2C burst.

COMMAND FORMAT

```
{"cmd": "servo", "pwms": [v0,v1,v2,v3,v4,v5,v6,v7,v8,v9,v10,v11]}
```

JOINT ORDER IN THE ARRAY

INDEX	JOINT	PCA CH	LIMITS (°)
0	FL_calf	0	150 → 0
1	FL_thigh	1	50 → 180
2	arm_a_2_1	2	180 → 50
3	RL_calf	4	0 → 150
4	RL_thigh	5	50 → 180
5	RL_hip	6	140 → 0
6	FR_calf	8	180 → 40
7	FR_thigh	9	140 → 0
8	FR_hip	10	30 → 150
9	RR_calf	12	40 → 180
10	RR_thigh	13	135 → 0
11	RR_hip	14	45 → 180

ALL JOINTS TO CENTER

```
echo '{"cmd": "servo", "pwms": [307,307,307,307,307,307,307,307,307,307,307,307]} ' \
| nc -q1 127.0.0.1 50007
```

INTERACTIVE TEST SCRIPT

```
sshpass -p 'arduinoverse' ssh -t -o StrictHostKeyChecking=no arduino@100.106.178.114 \
"python3 /tmp/servo_test.py"
```

COMMANDS INSIDE THE SCRIPT

```
center      # all joints → 307
joints      # show joint names and indices
set 0 200   # move joint 0 (FL_calf) to count 200
sweep 0     # slowly sweep joint 0 across full range
all 307     # set every joint to same value
imu         # read current IMU frame
quit
```

FROM PYTHON

```
import socket, json

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(('100.106.178.114', 50007))
pwms = [307] * 12      # all center
pwms[0] = 200          # FL_calf to 200
cmd = json.dumps({'cmd': 'servo', 'pwms': pwms}, separators=(',', ':')) + '\n'
s.sendall(cmd.encode())
print(json.loads(s.recv(1024)))
s.close()
```

8. PWM Values Explained

Values sent to the bridge are raw **PCA-9685 off-counts** (0–4095). The PCA-9685 runs at 50 Hz (20 ms period). Each count $\approx 4.88 \mu\text{s}$.

OFF-COUNT	PULSE WIDTH	SERVO POSITION
102	~0.5 ms	one mechanical limit
205	~1.0 ms	near one end
307	~1.5 ms	center (neutral)
410	~2.0 ms	near other end
512	~2.5 ms	other mechanical limit

The bridge passes values straight through to the PCA-9685. Your ROS 2 node is responsible for converting joint angles (degrees) to off-counts.

9. Check System Status

```
echo '{"cmd":"status"}' | nc -q1 127.0.0.1 50007
```

RESPONSE

```
{
  "pca9685_ok": true, // PCA-9685 found on I2C and awake
  "mpu6050_ok": true, // MPU-6050 found on I2C
  "ai_ok":      true, // PCA-9685 auto-increment bit set correctly
  "servo_calls": 42,   // servo commands MCU has executed
  "servo_reqs": 42,   // servo commands bridge received
  "ping_count": 1,
  "i2c_scan": 0       // 0=both OK; 1=PCA missing; 2=MPU missing; 3=both missing
}
```

servo_calls vs servo_reqs: if `servo_reqs` goes up but `servo_calls` stays flat, the MCU is not receiving commands — check the router connection with ping.

OTHER USEFUL COMMANDS

```
echo '{"cmd":"ping"}' | nc -q1 127.0.0.1 50007
echo '{"cmd":"scan_i2c"}' | nc -q1 127.0.0.1 50007
# scan_i2c expected: {"addrs":[64,104]} (0x40=PCA, 0x68=MPU)
```

WHO IS SENDING SERVO COMMANDS?

```
ss -tp sport = :50007    # active connections to the bridge
pgrep -a python3         # any servo_test.py still running?
docker ps               # Docker container should be stopped
journalctl -fu hardware-bridge.service # live command log
```


10. File Locations

~/poc/hardware_bridge.cpp	Bridge daemon source
~/poc/build/hardware_bridge	Compiled binary (what the service runs)
~/poc/CMakeLists.txt	Build config
~/ArduinoApps/hardware_bridge_app/sketch/sketch.ino	Active MCU sketch
/tmp/write_sketch4.py	Regenerates sketch.ino (run before flash)
/tmp/servo_test.py	Interactive servo test script
/etc/systemd/system/hardware-bridge.service	Systemd unit (auto-starts on boot)
/etc/systemd/system/arduino-router.service.d/20-baud.conf	Baud override — keep at 460800

FIX BAUD CONF (IF ROUTER EVER RESTARTS WRONG)

```
sudo tee /etc/systemd/system/arduino-router.service.d/20-baud.conf <<'EOF'
[Service]
ExecStart=
ExecStart=/usr/bin/arduino-router \
  --unix-port /var/run/arduino-router.sock \
  --serial-port /dev/ttyHS1 \
  --serial-baudrate 460800 \
  --after-ready "/usr/bin/gpioset -c /dev/gpiochip1 -t0 70=1"
EOF
sudo systemctl daemon-reload
# do NOT restart arduino-router after this
```